AI Project & CV Reviewer

System Analysis & Design Document

Version 2.0 | LangChain Multi-tool Agent | Grok (xAI)

Field	Details
Project Name	AI Project & CV Reviewer
Version	2.0 — LangChain Agent Architecture
Architecture	Multi-tool ReAct Agent (LangChain)
Backend	Flask + LangChain + Grok (xAI)
Deployment	HuggingFace Spaces (Docker)
Document Type	Requirements & System Analysis

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder analysis is the process of identifying all individuals, groups, or organizations that have an interest in the system, and understanding their needs, expectations, and potential influence on the project. For the AI Project & CV Reviewer system, four key stakeholder groups were identified.

Stakeholder	Role	Primary Needs	Influence
Developer / User	Primary End User	Upload code or CV and receive detailed AI-generated analysis, bug reports, and improvement suggestions	High — drives all functional requirements
Hiring Manager	Indirect Stakeholder	Receive well-structured, ATS-optimized CVs from candidates who used the system	Medium — influences CV analysis criteria
xAI (Grok)	LLM API Provider	API consumption within rate and usage limits; correct model usage	High — core dependency for AI responses
HuggingFace	Hosting Platform	Docker-based deployment compliant with Spaces SDK requirements; port 7860 exposure	High — deployment and availability
System Admin	Operations	Monitor API usage, rate limits, and session memory consumption	Medium — affects operational decisions

3.2 User Stories & Use Cases

User stories capture the functional needs of the system from the perspective of end users. Each story follows the standard format: As a [role], I want [goal] so that [benefit]. These stories directly drove the identification of functional requirements and the design of the LangChain agent tools.

ID	As a...	I want to...	So that...
US-01	Developer	upload a ZIP file of my project	I receive a comprehensive structured analysis covering code quality, bugs, and best practices
US-02	Developer	see detected bugs with severity levels	I can prioritize which issues to fix first
US-03	Developer	receive code improvement suggestions	I can directly apply the suggested fixes to my codebase
US-04	Job Seeker	upload my CV as a PDF	I receive ATS compatibility feedback and scoring before applying for jobs
US-05	User	ask follow-up questions after analysis	I can clarify specific parts of the report without re-uploading the file

US-06	User	have the system remember my conversation	I do not need to repeat context in every message
US-07	User	delete my session when done	my data is not retained unnecessarily
US-08	Developer	have the system search for relevant docs	the analysis references up-to-date library documentation

3.3 Functional Requirements

Functional requirements define the specific behaviors and capabilities the system must provide. They describe what the system does in response to particular inputs or conditions. The following table lists all functional requirements for the AI Project & CV Reviewer system, derived directly from the stakeholder analysis and user stories above.

ID	Feature	Description	Priority
FR-01	File Upload	System shall accept PDF, ZIP, and plain source code files up to 10 MB via a multipart form POST request	Must Have
FR-02	Text Extraction	System shall automatically extract readable text from PDF pages using PyPDF2 and from ZIP archives by iterating all supported source files	Must Have
FR-03	Content Detection	System shall automatically detect whether uploaded content is a code project or a CV based on keyword scoring	Must Have
FR-04	Code Structure Tool	The agent shall use code_structure_analyzer to identify language, line count, imports, classes, async usage, and test presence	Must Have
FR-05	Bug Detection Tool	The agent shall use bug_detector to identify critical security issues including debug mode, hardcoded secrets, and SQL injection patterns	Must Have
FR-06	Best Practices Tool	The agent shall use best_practices_checker to evaluate adherence to PEP8, Flask conventions, and production readiness standards	Must Have
FR-07	Code Improver Tool	The agent shall use code_improver to generate ready-to-use improved code for identified issues upon request	Should Have
FR-08	CV Analysis Tool	The agent shall use cv_analyzer to calculate ATS score, identify missing sections, and list detected technical skills	Must Have
FR-09	Web Search Tool	The agent shall use web_search via DuckDuckGo to retrieve current documentation and best practices relevant to the project	Should Have
FR-10	Session Management	System shall create a unique UUID session per upload, maintain conversation memory, and allow session deletion via DELETE endpoint	Must Have
FR-11	Chat Follow-up	System shall accept follow-up questions tied to an existing session and respond using full conversation history as context	Must Have

FR-12	History Retrieval	System shall expose a GET endpoint returning the full user and assistant message history for any active session	Should Have
FR-13	Rate Limiting	System shall enforce a maximum of 20 requests per IP address per hour across all endpoints	Must Have
FR-14	Health Check	System shall expose a GET /health endpoint returning service status and active model name	Nice to Have

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system. They specify how the system should perform its functions rather than what those functions are. These requirements cover performance, security, usability, reliability, maintainability, and deployment portability.

Category	Requirement	Acceptance Criteria
Performance	Agent response time	Full analysis response must be delivered within 30 seconds under normal load
Performance	File processing speed	Text extraction from files up to 10 MB must complete within 3 seconds
Scalability	Concurrent sessions	System must support at least 20 simultaneous active sessions without degradation
Security	Rate limiting	Each IP is restricted to 20 requests per hour; excess requests return HTTP 429
Security	No secret exposure	API keys must be loaded exclusively from environment variables, never hardcoded
Security	File validation	Only whitelisted extensions are accepted; unsupported types return HTTP 415
Reliability	Error handling coverage	All endpoints must catch exceptions and return structured JSON error messages
Reliability	Agent failure recovery	Agent must handle LLM timeouts gracefully with a user-readable error response
Usability	Arabic language responses	All AI-generated analysis and chat responses must be in formal technical Arabic
Usability	Structured output	Analysis reports must use Markdown headings, tables, and sections for readability
Maintainability	Modular architecture	Each concern (tools, prompts, config, utils) must reside in a dedicated module
Maintainability	Configuration centralization	All constants must be defined in config/settings.py with no magic numbers in routes
Portability	Docker containerization	Application must run inside a Docker container compatible with HuggingFace Spaces SDK
Portability	Environment independence	No hardcoded host paths or OS-specific dependencies; must run on Linux and macOS

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem Statement

Software developers frequently struggle to identify code quality issues, security vulnerabilities, and deviations from best practices without spending significant time on manual code review. Similarly, job seekers often submit CVs that fail to pass Applicant Tracking System (ATS) filters due to poor formatting, missing keywords, or lack of quantified achievements — without receiving actionable feedback on how to improve them.

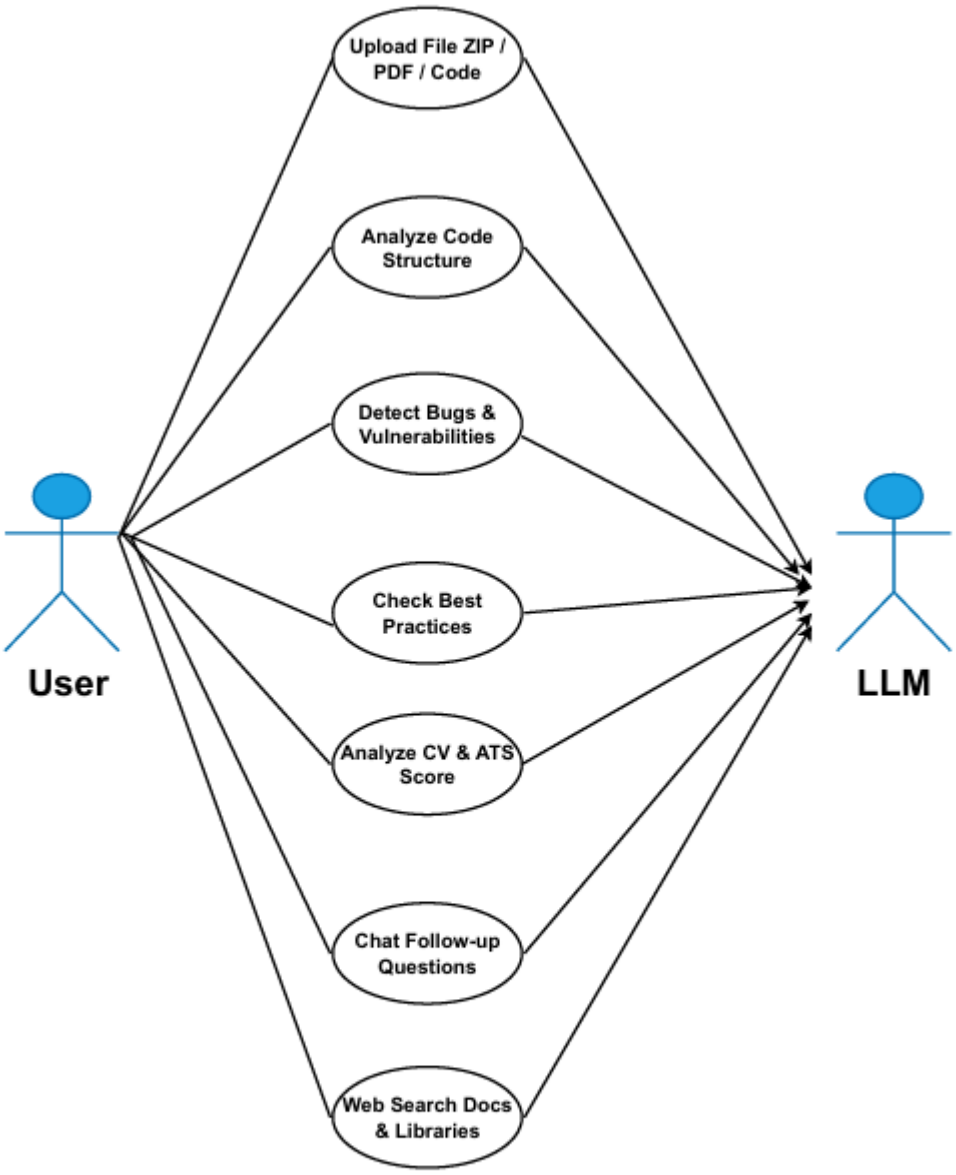
Existing tools for code review are either too generic, require expensive subscriptions, or do not provide contextual follow-up capabilities. CV review tools similarly lack technical depth and do not provide ATS-specific scoring tied to real job requirements.

Project Objectives

- Automate the end-to-end analysis of code projects by orchestrating multiple specialized AI tools through a LangChain ReAct Agent.
- Detect security vulnerabilities, code smells, and best practice violations in uploaded source code, and generate ready-to-apply code improvements.
- Provide comprehensive CV analysis including ATS compatibility scoring, skill gap identification, and professional summary generation.
- Enable contextual multi-turn conversations scoped to the uploaded project, allowing users to ask specific follow-up questions without losing context.
- Deploy the system as a rate-limited, containerized REST API on HuggingFace Spaces, accessible to any HTTP client or frontend application.

4.2 Use Case Diagram & Descriptions

The system involves two primary actors: the User (developer or job seeker) who interacts with the system through the REST API, and the Grok LLM which acts as the reasoning engine behind the LangChain agent. The Use Case Diagram for this system is provided separately as a Mermaid diagram. Below are detailed descriptions of each identified use case.


Upload File ZIP / PDF / Code
Analyze Code Structure
Detect Bugs & Vulnerabilities
Check Best Practices
Analyze CV & ATS Score
Chat Follow-up Questions
Web Search Docs & Libraries
User
LLM

Use Case	Actor	Precondition	Main Flow	Postcondition
Upload & Analyze File	User	User has a valid code file or CV ready	User sends POST /AI with file. System validates, extracts text, detects type, creates session, runs agent with appropriate tools, returns analysis JSON	Session created; analysis stored in memory
Detect Code Bugs	Agent + Grok	Code content extracted successfully	Agent calls bug_detector tool with code. Tool scans for debug mode, hardcoded secrets, SQL patterns. Returns structured findings.	Bug report included in final analysis
Analyze CV	Agent + Grok	CV content detected by keyword scoring	Agent calls cv_analyzer. Tool calculates ATS score, identifies GitHub/LinkedIn presence, lists tech skills, provides recommendations.	CV report with score returned to user
Chat Follow-up	User	Valid session_id exists with active memory	User sends POST /chat with message. Agent receives message with full memory context. Calls tools if needed. Returns contextual reply.	New messages appended to session memory
Web Search	Agent	Agent determines documentation needed	Agent calls web_search tool with query. DuckDuckGo returns results. Agent incorporates results into reasoning.	Search results included in agent reasoning
Delete Session	User	Session exists in agent_executors dict	User sends DELETE /session/{id}. System removes AgentExecutor and project context from memory.	Session fully removed; memory freed

4.3 Software Architecture

The system follows a Layered Architecture pattern with clear separation of concerns across five distinct layers. This architecture was chosen over alternatives like MVC or Microservices because the system is a single-process application where logical separation is more important than physical service distribution. Each layer has a single responsibility and communicates only with adjacent layers.

Layer	Components	Responsibility
Presentation Layer	app.py (Flask REST API)	Handles HTTP routing, request validation, file size and extension checks, rate limit enforcement, and JSON response formatting. Contains no business logic.
Orchestration Layer	agent_builder.py (AgentBuilder + AgentExecutor)	Constructs and manages the LangChain ReAct agent for each session. Responsible for wiring together the LLM, tools, memory, and prompt. Isolates all LangChain-specific logic.
Business Logic Layer	tools/ (6 Specialized Tool Functions)	Each tool encapsulates a specific analysis capability: structure analysis, bug detection, best practices checking, code improvement, CV

		analysis, and web search. Tools are stateless and independently testable.
Infrastructure Layer	utils/ + config/ (FileHandler, RateLimiter, Settings)	Provides cross-cutting concerns: text extraction from multiple file types, rate limiting, and centralized configuration constants. No AI logic exists at this layer.
External Services	Grok LLM API + DuckDuckGo Search	Third-party services consumed by the orchestration and tool layers. The system is designed so that the LLM provider can be swapped by changing a single configuration value in settings.py.

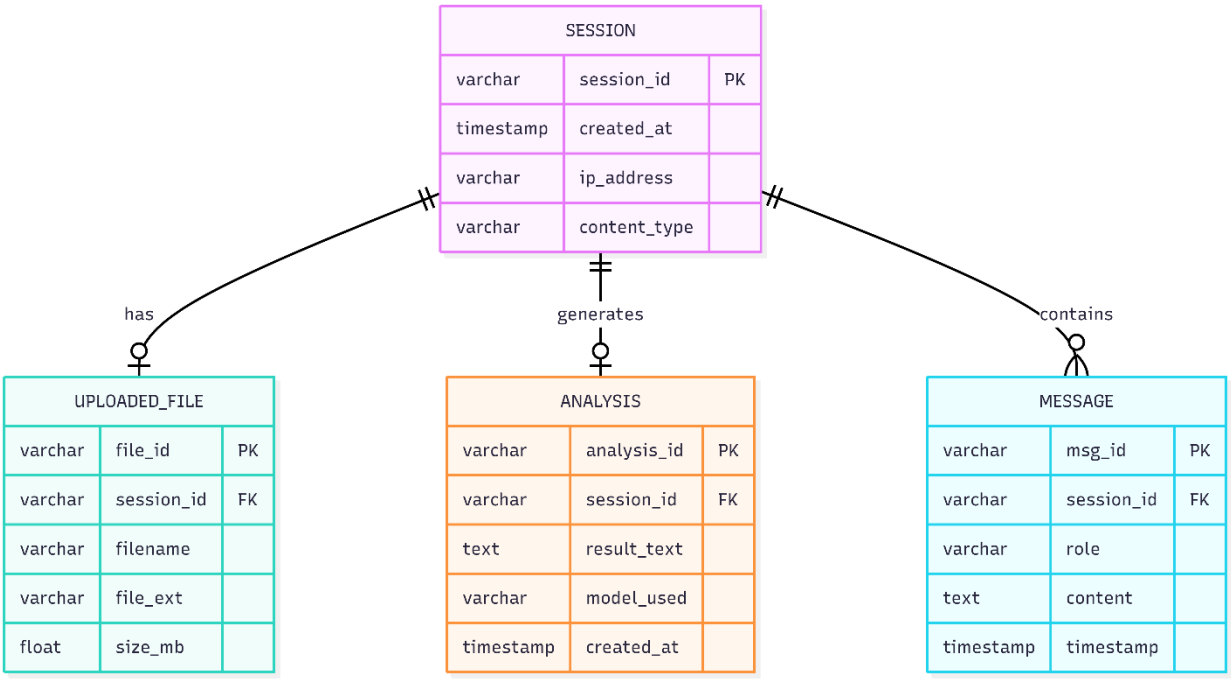
The agent uses the ReAct (Reasoning + Acting) pattern, where the LLM alternates between reasoning steps (deciding which tool to call next) and acting steps (calling the tool and observing results). This loop continues until the LLM determines it has sufficient information to generate a final answer, or until the maximum iteration count of 6 is reached.

4.2 Database Design & Data Modeling

The current implementation uses in-memory Python dictionaries for session and history storage, which is appropriate for the initial deployment on HuggingFace Spaces where a database service is not available. However, the logical data model below represents the relational structure that would be implemented in a production Redis or PostgreSQL-backed version of the system.

ER Diagram Description

The ER Diagram (provided separately as a Mermaid diagram) contains four entities: SESSION, UPLOADED_FILE, ANALYSIS, and MESSAGE. The SESSION entity is the central entity to which all others relate. Each session has exactly one uploaded file and generates exactly one analysis result. A session can contain many messages, forming the conversation history. The diagram uses standard crow's foot notation to show cardinality.



Logical Schema

SESSION Table

The SESSION table is the root entity representing one user interaction cycle. The session_id is a UUID version 4 string generated at upload time and serves as the primary key across all related tables. The content_type column stores either 'code' or 'cv' as detected by the keyword scoring algorithm in FileHandler. The ip_address column supports rate limiting and abuse monitoring.

Column	Type	Constraint	Description
session_id	VARCHAR(36)	PRIMARY KEY	UUID v4 generated at upload time
created_at	TIMESTAMP	NOT NULL	Session creation timestamp
ip_address	VARCHAR(45)	NOT NULL	Client IP for rate limiting (supports IPv6)
content_type	VARCHAR(10)	NOT NULL	Detected type: 'code' or 'cv'

UPLOADED_FILE Table

The UPLOADED_FILE table records metadata about the file submitted by the user. The actual text content is not stored in a relational column due to its potentially large size; in the Redis production model it would be stored as a separate key-value entry. The size_mb column supports enforcement of the 10 MB upload limit at the application layer.

Column	Type	Constraint	Description
file_id	VARCHAR(36)	PRIMARY KEY	UUID v4 for the file record
session_id	VARCHAR(36)	FOREIGN KEY	References SESSION(session_id)
filename	VARCHAR(255)	NOT NULL	Original filename as submitted
file_ext	VARCHAR(10)	NOT NULL	File extension e.g. .py .pdf .zip
size_mb	FLOAT	NOT NULL	File size in megabytes

ANALYSIS Table

The ANALYSIS table stores the final output generated by the LangChain agent. The result_text column holds the complete Markdown-formatted analysis report. The model_used column records the LLM model identifier to support future A/B testing and audit trails. This table has a one-to-one relationship with SESSION because each upload produces exactly one initial analysis.

Column	Type	Constraint	Description
analysis_id	VARCHAR(36)	PRIMARY KEY	UUID v4 for the analysis record
session_id	VARCHAR(36)	FOREIGN KEY	References SESSION(session_id)
result_text	TEXT	NOT NULL	Full Markdown analysis from agent
model_used	VARCHAR(50)	NOT NULL	LLM model name e.g. grok-3-mini
created_at	TIMESTAMP	NOT NULL	Timestamp of analysis completion

MESSAGE Table

The MESSAGE table stores the full conversation history for each session. Each row represents one turn in the conversation. The role column follows the OpenAI message format with values 'user' or 'assistant'. This table supports the chat follow-up feature and enables the system to reconstruct the full conversation context for each agent invocation. It has a one-to-many relationship with SESSION as a session can accumulate an unlimited number of messages.

Column	Type	Constraint	Description
msg_id	VARCHAR(36)	PRIMARY KEY	UUID v4 for the message record
session_id	VARCHAR(36)	FOREIGN KEY	References SESSION(session_id)
role	VARCHAR(10)	NOT NULL	Message author: 'user' or 'assistant'
content	TEXT	NOT NULL	Full message text content
timestamp	TIMESTAMP	NOT NULL	When the message was created

4.3 Data Flow & System Behavior

Data Flow Diagram (DFD)

The DFD (provided separately as a Mermaid diagram) illustrates how data moves through the system at two levels. At the context level, the system has two external entities: the User who provides files and receives reports, and the Grok API which receives queries and returns AI-generated responses.

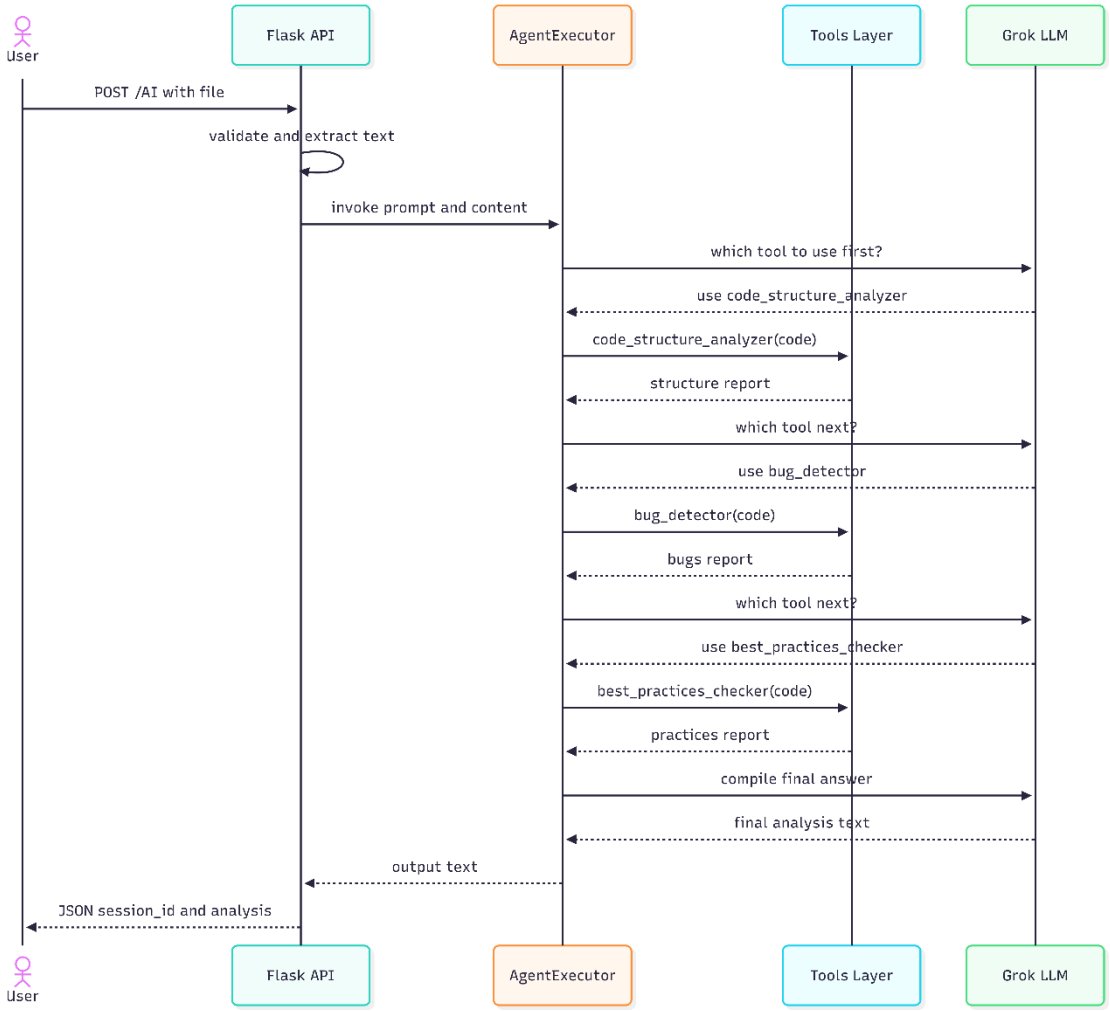
At the detailed level, four internal processes handle the data flow. Process P1 (Upload & Parse File) receives the raw file from the user and produces extracted plain text. Process P2 (Detect Content Type) classifies the text as either 'code' or 'cv' using keyword scoring and routes it to P3 accordingly. Process P3 (Run Agent Tools) is the core processing stage where the LangChain agent iteratively calls tools and the Grok API until a complete analysis is available. It also writes to data stores D1 (Session Store) and D2 (Chat History). Process P4 (Generate Report) formats the agent output into a structured JSON response returned to the user.

Sequence Diagram Description

The Sequence Diagram (provided separately as a Mermaid diagram) describes the interaction between five participants in the primary use case: User, Flask API, AgentExecutor, Tools Layer, and Grok LLM.

The interaction begins when the User sends a POST /AI request with an attached file. The Flask API validates the file, extracts its text content, and invokes the AgentExecutor with an initial prompt containing the extracted content. The AgentExecutor enters the ReAct loop by querying the Grok LLM to decide which tool to call first. Grok responds by selecting code_structure_analyzer. The executor calls this tool and receives a structured report of the code's architecture.

The loop continues: the agent queries Grok again, which selects bug_detector. The tool scans the code and returns a list of identified issues. On the third iteration, Grok selects best_practices_checker, which evaluates the code against PEP8 and Flask conventions. Once sufficient information is gathered, Grok compiles all tool outputs into a comprehensive final answer. This answer is returned through the AgentExecutor to the Flask API, which packages it into a JSON response containing the session_id and the full analysis text.

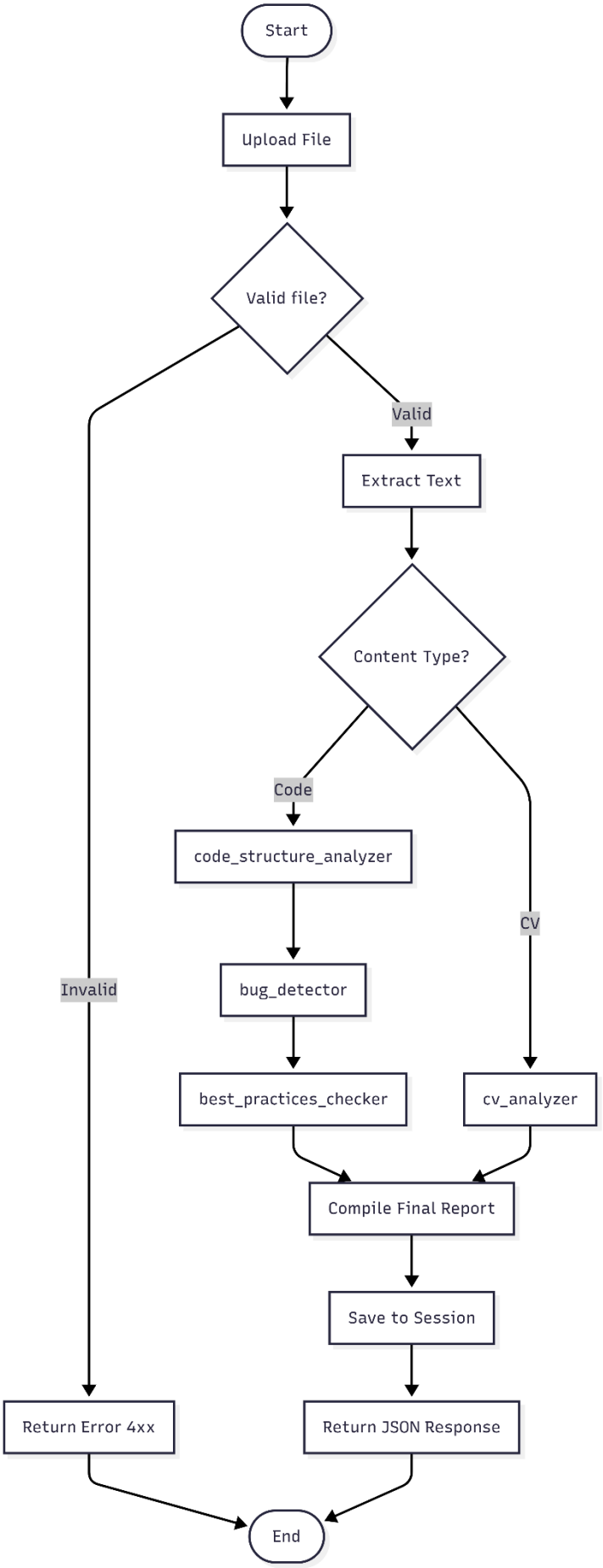


Activity Diagram Description

The Activity Diagram (provided separately as a Mermaid diagram) visualizes the end-to-end workflow from the moment a user initiates an upload to the delivery of the analysis response.

The workflow begins at a start node. The first activity is file upload, followed immediately by a decision node that checks whether the file is valid in terms of size and extension. If invalid, the workflow branches to an error response activity and terminates. If valid, text extraction occurs, followed by a second decision node for content type detection.

The content type decision creates two parallel branches. The 'Code' branch executes three sequential activities: code_structure_analyzer, then bug_detector, then best_practices_checker. The 'CV' branch executes a single activity: cv_analyzer. Both branches converge at a synchronization bar before proceeding to the Compile Final Report activity. The workflow then saves the session and returns the JSON response before reaching the end node.


Start
Upload File
Valid file?
Valid
Invalid
Extract Text
Content Type?
Code
CV
code_structure_analyzer
bug_detector
best_practices_checker
cv_analyzer
Compile Final Report
Save to Session
Return JSON Response
Return Error 4xx
End

State Diagram Description

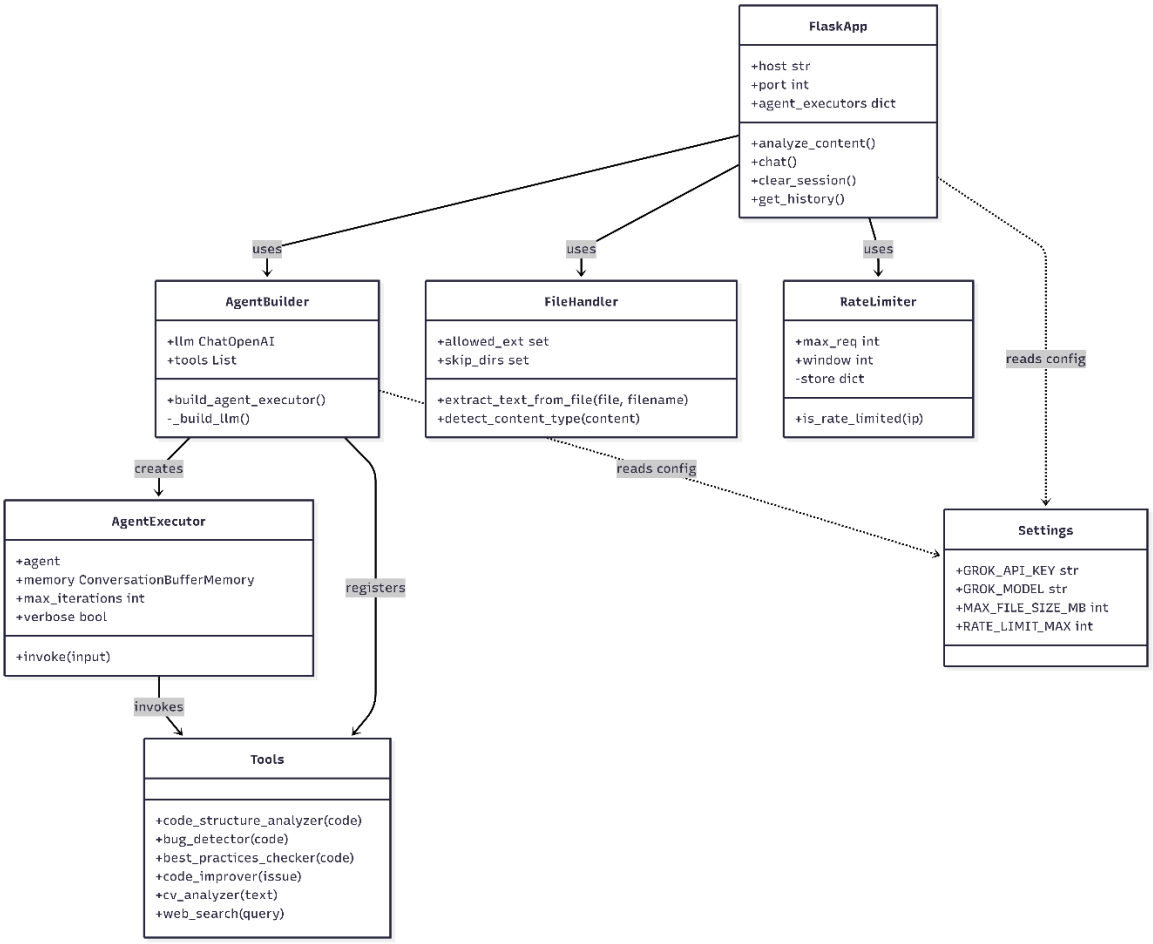
The State Diagram (provided separately as a Mermaid diagram) models the lifecycle of a single user session object, showing all possible states and the events that trigger transitions between them.

A session begins in the IDLE state immediately after the system starts. When a user uploads a file, the session transitions to UPLOADING. Once the file bytes are received by the server, the session moves to VALIDATING where size and extension checks are performed. If validation fails — for example due to an unsupported file type or a file exceeding 10 MB — the session transitions to the ERROR state, from which it can return to IDLE if the user retries.

If validation succeeds, the session enters ANALYZING, where the LangChain agent processes the content. A timeout or API failure during analysis also leads to ERROR. Successful analysis completion transitions the session to READY, indicating that the initial report has been delivered and the session is ready for follow-up questions. While in READY, sending a message transitions the session to CHATTING, and receiving the agent reply returns it to READY. At any point while READY, the user may delete the session, transitioning it to TERMINATED, after which it reaches the final state and is removed from memory.

Class Diagram Description

The Class Diagram (provided separately as a Mermaid diagram) defines the structural design of the system by showing six main classes, their attributes, methods, and inter-class relationships.



FlaskApp

The entry point of the application. Owns the agent_executors dictionary (session_id to AgentExecutor mapping) and the project_contexts dictionary. Exposes five public methods corresponding to the five API endpoints. Delegates all business logic to AgentBuilder, FileHandler, and RateLimiter.

AgentBuilder

Responsible for constructing and configuring AgentExecutor instances. Holds a reference to the shared ChatOpenAI LLM instance and the ALL_TOOLS list. The build_agent_executor() method creates a new AgentExecutor with a fresh ConversationBufferMemory for each session. The private _build_llm() method reads credentials from Settings.

AgentExecutor

A LangChain class that manages the ReAct loop. It holds the agent (reasoning component), memory (conversation history), max_iterations limit, and verbose flag. The invoke(input) method runs the full reasoning-acting loop and returns the final output dictionary.

Tools

A logical class representing the six tool functions defined in the tools/ package. Each method is decorated with @tool from LangChain, making it discoverable by the AgentExecutor. Tools are stateless pure functions that accept string inputs and return string outputs.

FileHandler

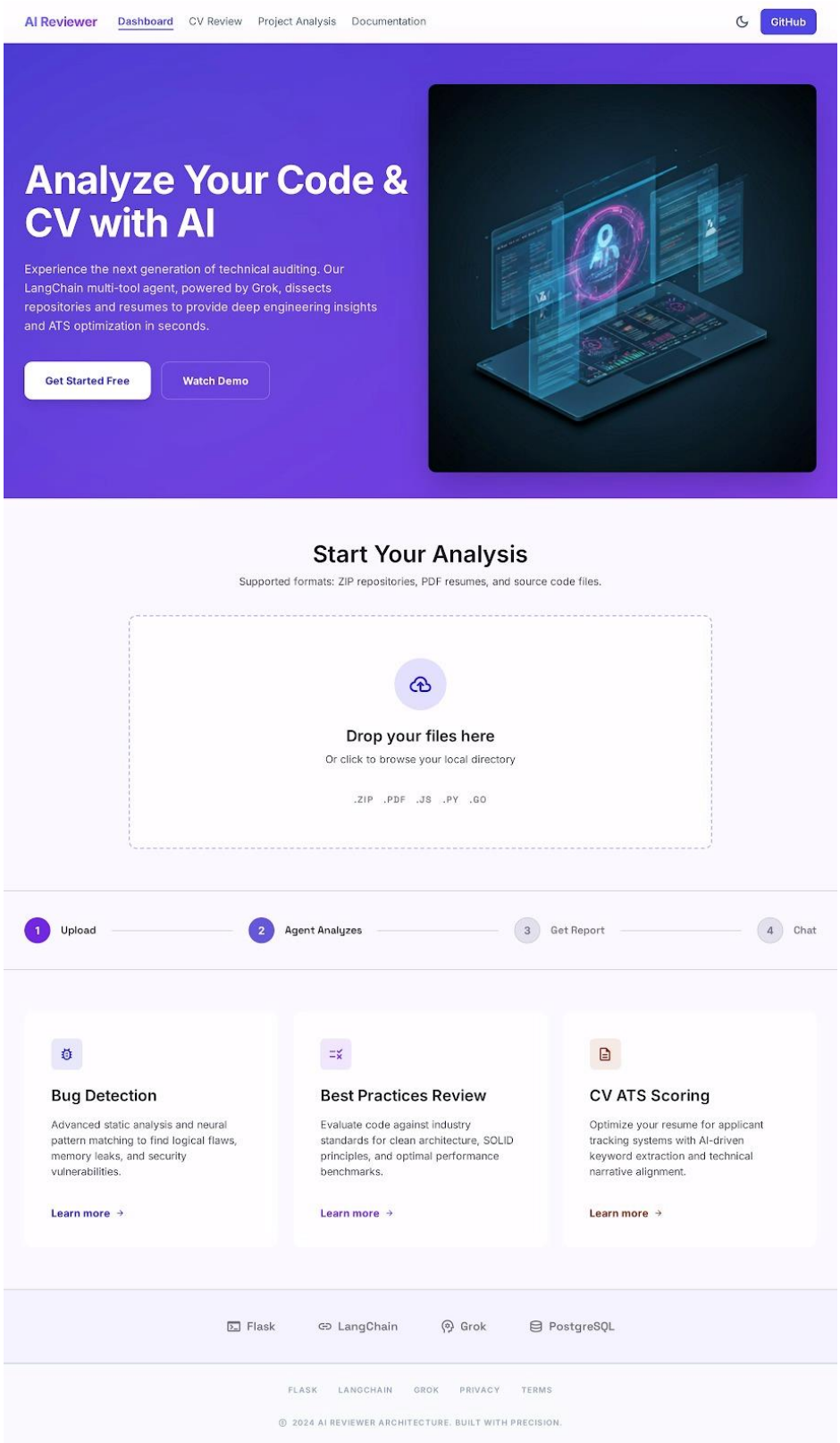
Handles all file I/O operations. The extract_text_from_file() method dispatches to appropriate handlers based on file extension. The detect_content_type() method uses keyword scoring to classify content. Both methods read configuration from the Settings class.

RateLimiter

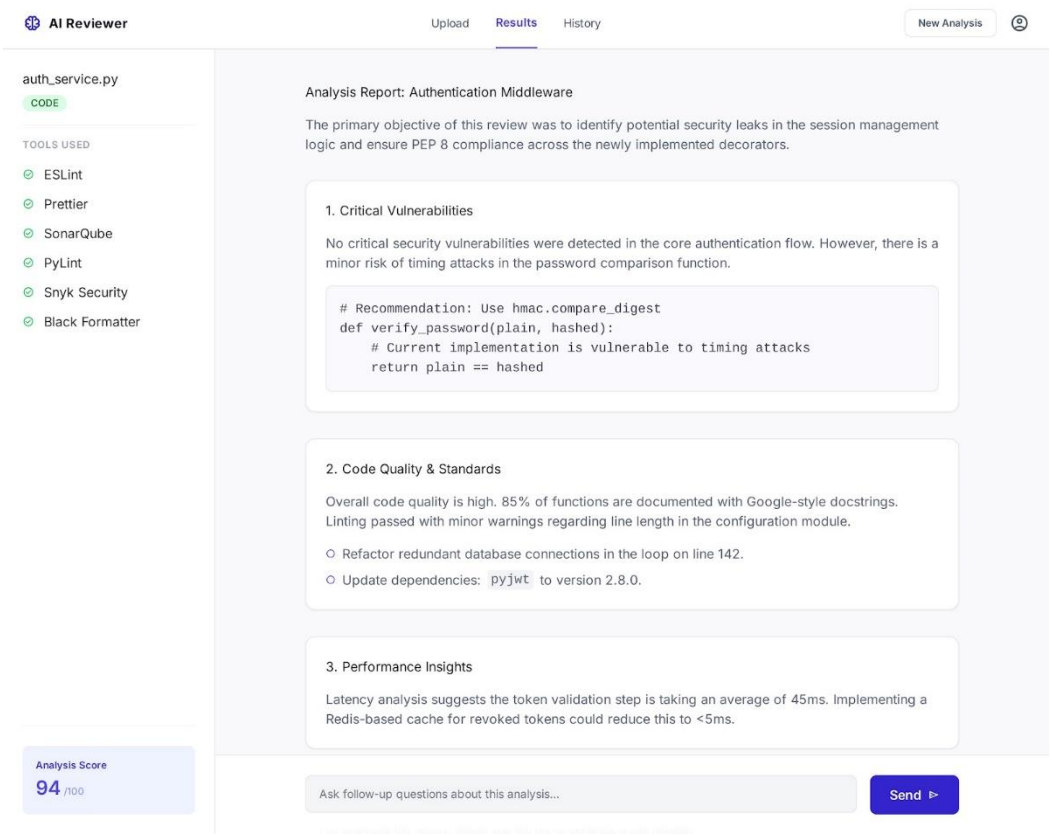
Manages per-IP request throttling using an in-memory defaultdict of timestamps. The is_rate_limited() method checks and updates the rate limit store atomically, returning True if the limit has been exceeded and False otherwise, registering the current request in the latter case.

The relationship structure shows that FlaskApp depends on AgentBuilder, FileHandler, and RateLimiter. AgentBuilder creates AgentExecutor instances and registers the Tools with them. AgentExecutor invokes Tools during the ReAct loop. Both FlaskApp and AgentBuilder have a dependency on Settings for configuration constants. This design ensures that no circular dependencies exist and that each class can be unit tested in isolation by mocking its dependencies.

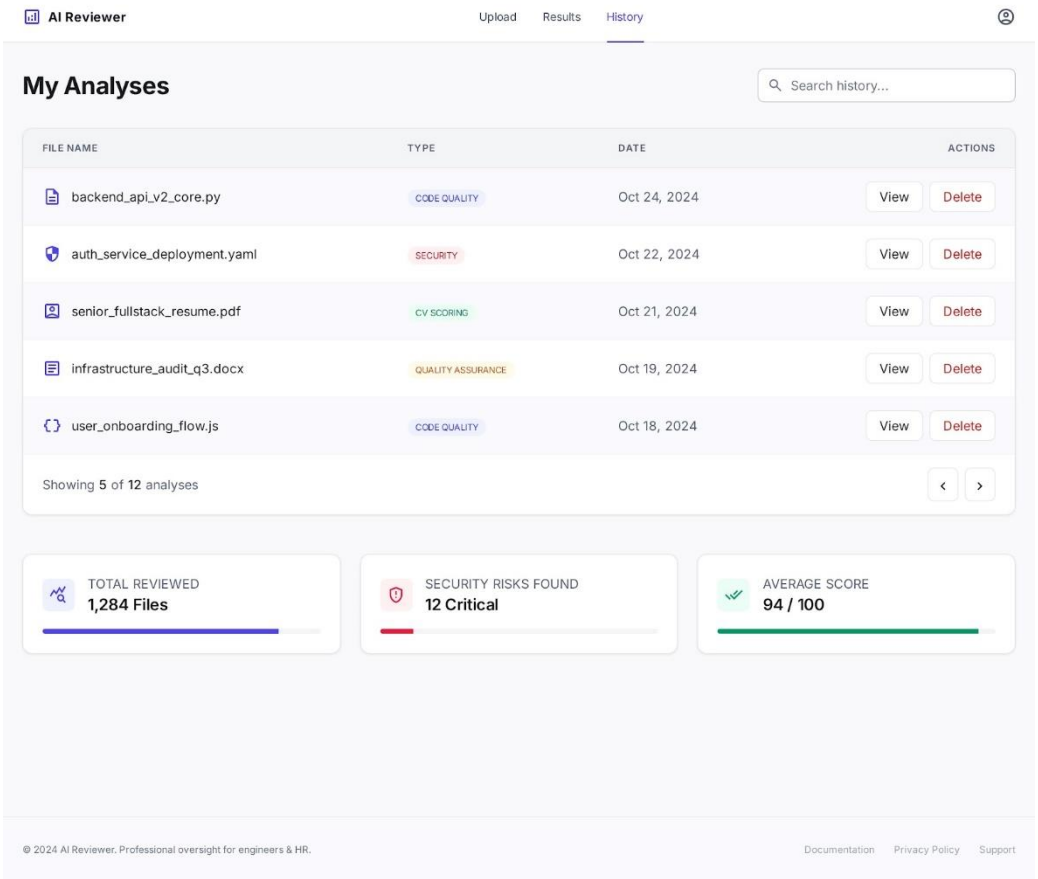
4.4 UI/UX Design & Prototyping



Landing Page: The homepage of "AI Reviewer", a platform that analyzes code and CVs using AI, featuring a file upload area and highlights of key features like bug detection and CV scoring.



Results Page: Displays a detailed analysis report after uploading a file, covering security vulnerabilities, code quality, and performance insights, with an overall score of 94/100.



History Page: Shows a log of all previously analyzed files with their type and date, along with overall stats like total files reviewed, critical security risks found, and average score